SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library

CO2: *Apply image enhancement and segmentation techniques for extracting meaningful regions from images.*

Student Name	M.DEVI SRILATHA	**Register Number**	192525236
Department / Section	Information Technology	**Academic Year / Semester**	2025-2026
Faculty Name	Dr.P.Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50)

Scenario – Smart Document Scanner for the College Library

The college library wants to digitize its old printed question papers and lecture notes. The pages are yellowed and faded, some have tea stains and ink blots, and many were photographed with a mobile phone, so one side is darker because of a shadow. The library wants a simple "smart scanner" program (like the scanner apps on your phone) that (1) enhances each page so the text is clear and easy to read, and (2) segments the page: separates the text from the paper

TASK 1 – BASIC GREY LEVEL TRANSFORMATIONS

Compare Point-Processing Techniques

1. Introduction

Old question papers often contain faded ink, yellowed paper and uneven illumination, making text difficult to read. Point-processing techniques improve image quality by modifying individual pixel intensities.

The transformation is represented as:

$s = T(r)$

Where r is the input intensity and s is the output intensity.

2. Techniques Used

A. Negative + Log Transformation

- Negative: $s = 255 - r$
- Log: $s = c \log(1 + r)$
- Expands darker intensity levels.
- Implemented using cv2.LUT().

B. Power-Law (Gamma) Transformation

Formula: $s = 255 \times (r/255)^{\gamma}$

- $\gamma > 1$: Darkens the image.
- $\gamma < 1$: Brightens the image.
- $\gamma = 2$ is selected to darken faded text.

C. Piecewise-Linear Contrast Stretching

- Stretches the intensity range 90–180 to 0–255.
- Improves text-to-paper contrast.
- Values outside the selected range are clipped.

3. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("page1.jpg", 0)

# Negative + Log
def neg_log(img):
    c = 255 / np.log(256)
    lut = np.array([
        c * np.log1p(255-r) for r in range(256)
    ])
    return cv2.LUT(img, (255-lut).clip(0,255).astype(np.uint8))

# Gamma Transformation
def gamma(img, g=2):
    lut = (255*(np.arange(256)/255)**g).astype(np.uint8)
    return cv2.LUT(img, lut)

# Contrast Stretching
def stretch(img, r1=90, r2=180):
    r = np.arange(256)
    lut = np.clip((r-r1)*255/(r2-r1),0,255)
    return cv2.LUT(img, lut.astype(np.uint8))

A = neg_log(img)
B = gamma(img, 2)
C = stretch(img)

images = [img, A, B, C]
titles = ["Original", "Negative + Log",
          "Gamma", "Contrast Stretching"]

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

4. Experimental Comparison

Text contrast is calculated as:

Text Contrast = Mean Paper Intensity − Mean Ink Intensity

Technique	Text Contrast	Clipping (%)	Time (ms)
Original	39.5	0	0
Negative + Log	23.0	0	3.3
Gamma (2.0)	50.4	0.03	2.1
Contrast Stretching	69.9	62.74	2.4

Observation:

- Negative + Log reduces text contrast.
- Gamma improves contrast with minimal clipping.
- Contrast stretching provides the highest contrast but loses some image details.

5. Bit-Plane Slicing

An 8-bit grayscale image contains eight bit planes (7 to 0).

Bit Plane	Text Visibility
7	0.00
6	0.70
5	0.23
4	0.20
3–0	0.00–0.01

Observation: Bit 6 preserves most text information, while the lower bit planes are mainly affected by image noise.

6. Gamma and Contrast Stretching Analysis

Applying the same gamma value to light and dark pages produces different results.

For $\gamma = 2$:

- Light-page contrast increases from 33.8 to 55.0.
- Dark-page contrast increases only from 31.1 to 34.9.
- The dark page becomes excessively dark.

Contrast stretching can remove faint pencil marks when the upper control point is below the paper intensity.

Therefore, parameters must be selected according to the individual page histogram.

7. Pretrained Model Comparison

Pretrained document-enhancement models such as DocEnTr and DE-GAN can improve faded text and remove shadows.

They are compared using:

- Text readability.
- Faint stroke preservation.
- Shadow removal.
- Processing time.
- Artificially altered strokes.

Limitation: The pretrained model was not executed. Therefore, actual measured comparison results are not available.

TASK 2 – HISTOGRAM PROCESSING

Compare Histogram-Based Enhancement

1. Introduction

Scanned question papers from different years have variations in brightness, contrast and illumination. Histogram processing improves image quality by modifying the distribution of pixel intensities.

Three techniques are compared:

A. Global Histogram Equalization

- Uses the cumulative distribution function (CDF) of the entire image.
- Improves overall contrast.
- OpenCV function: cv2.equalizeHist().

B. Histogram Matching

- Matches the histogram of an input page with a clean reference page.
- Produces consistent brightness and contrast.
- Implemented using CDF-to-CDF mapping.

C. CLAHE (Contrast Limited Adaptive Histogram Equalization)

- Divides the image into smaller tiles.
- Enhances each tile independently.
- Limits noise amplification using a clip limit.
- OpenCV function: cv2.createCLAHE().

2. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("page1.jpg", 0)
ref = cv2.imread("reference.jpg", 0)

# A. Global Histogram Equalization
A = cv2.equalizeHist(img)

# B. Histogram Matching
def hist_match(img, ref):
    c1 = np.cumsum(np.bincount(img.ravel(), minlength=256))
    c2 = np.cumsum(np.bincount(ref.ravel(), minlength=256))

    c1 = c1 / img.size
    c2 = c2 / ref.size

    lut = np.searchsorted(c2, c1)
    return cv2.LUT(img, lut.clip(0,255).astype(np.uint8))

B = hist_match(img, ref)

# C. CLAHE
clahe = cv2.createCLAHE(
    clipLimit=4.0,
    tileGridSize=(16,16)
)
C = clahe.apply(img)

# Display results
images = [img, A, B, C]
titles = ["Original", "Global HE",
          "Histogram Matching", "CLAHE"]

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Experimental Comparison

Image contrast and entropy are used to evaluate enhancement quality.

Technique	Page	Text Contrast	Entropy
Original	Light	33.8	5.47
Global HE	Light	122.9	5.36
Histogram Matching	Light	22.4	3.50
CLAHE	Light	63.6	6.61
Original	Normal	35.7	5.43
Global HE	Normal	124.2	5.32
Histogram Matching	Normal	23.2	3.54
CLAHE	Normal	63.7	6.62
Original	Shadowed	46.5	6.92
Global HE	Shadowed	77.6	6.83
Histogram Matching	Shadowed	9.4	3.47
CLAHE	Shadowed	69.4	7.26

Processing Time:

Technique	Time per Page
Global HE	3.5 ms
Histogram Matching	45 ms
CLAHE	20 ms

Observation:

- Global HE improves contrast but amplifies paper noise.
- Histogram matching provides consistent intensity distribution but may reduce text contrast.
- CLAHE improves local contrast and shadowed text visibility.

4. Local Statistics and CLAHE Parameters

Global histogram equalization uses one mapping for the entire page. Therefore, it cannot properly correct different illumination levels.

In the shadowed page:

- Shadow-side local mean = 85.
- Lit-side local mean = 191.
- Local standard deviation = 16 on both sides.

CLAHE Parameter Comparison

Tile Size	Clip Limit	Text Contrast	Evenness
4×4	2	66.9	0.46
8×8	2	68.2	0.56
16×16	2	66.1	0.57
16×16	4	83.5	0.71
16×16	8	107.5	0.83

Observation: A clip limit of 4 with 16×16 tiles provides a good balance between contrast improvement and noise control.

5. Special Cases and Limitations

Global HE on 90% Blank Paper:

When most of the page contains blank paper, histogram equalization stretches the dominant background intensity range. This amplifies paper grain and makes the background appear noisy.

Excessively High CLAHE Clip Limit:

A very high clip limit amplifies noise in mobile-phone photographs. Background speckles may become stronger and reduce text readability.

Tile Boundaries:

Small tiles may produce uneven local enhancement. Interpolation between tiles helps reduce visible boundaries.

6. Final Decision

CLAHE with a clip limit of 4 and tile size of 16×16 is selected.

- Global HE amplifies blank-paper noise and cannot properly correct spatial shadows.
- Histogram matching reduces contrast when the reference histogram is unsuitable.
- CLAHE improves local contrast and shadow visibility while controlling noise.

7. Conclusion

Histogram processing improves the readability of faded documents. Global histogram equalization provides overall contrast enhancement, histogram matching produces consistent intensity distribution, and CLAHE improves local contrast.

Therefore, CLAHE is selected as the most suitable technique for documents containing uneven illumination and shadows.

TASK 3 – Spatial Filtering and Image Smoothing: Compare Smoothing Filters

1. Introduction

Scanned pages may contain fine grain noise and dust specks. Smoothing filters remove this noise while preserving thin text strokes (2–4 pixels).

Three filters are compared:

Filter	Basic Idea	OpenCV Function
Box Filter	Replaces pixels with neighbourhood average.	cv2.blur()
Gaussian Filter	Uses a weighted average, giving more importance to centre pixels.	cv2.GaussianBlur()
Median Filter	Replaces pixels with the neighbourhood median.	cv2.medianBlur()

2. Implementation and Experiment

Five clean pages were used. Artificial noise was added with Gaussian grain ($\sigma = 6$) and 1% salt-and-pepper noise. The filters were tested using 3×3, 5×5 and 7×7 kernels.

Python Code

```
import cv2
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim

img = cv2.imread("noisy.jpg", 0)
clean = cv2.imread("clean.jpg", 0)

box = cv2.blur(img, (3, 3), borderType=cv2.BORDER_REFLECT)
gaussian = cv2.GaussianBlur(img, (3, 3), 0)
median = cv2.medianBlur(img, 3)

images = [img, box, gaussian, median]
titles = ["Noisy", "Box", "Gaussian", "Median"]
```

```
for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

    print(titles[i], "PSNR:", psnr(clean, images[i]),
          "SSIM:", ssim(clean, images[i]))

plt.tight_layout()
plt.show()
```

3. Comparison of PSNR, SSIM and Processing Time

The following values are from the supplied experiment results.

Filter	Kernel	PSNR (dB)	SSIM	Text Dice	Time (ms)
None	–	25.7	0.50	0.18	0.0
Box	3×3	33.4	0.81	0.52	3.1
Box	5×5	33.7	0.88	0.53	3.8
Box	7×7	32.9	0.89	0.51	7.5
Gaussian	3×3	33.2	0.80	0.64	2.0
Gaussian	5×5	34.4	0.86	0.58	3.1
Gaussian	7×7	34.5	0.90	0.55	9.1
Median	3×3	37.4	0.91	0.87	2.8
Median	5×5	35.6	0.93	0.73	23.6
Median	7×7	33.4	0.91	0.56	245.4

Observations:

- Box and Gaussian filters reduce grain but may spread dust into grey patches.
- Median filtering removes isolated dust pixels effectively.
- Median 3×3 preserves small characters better.
- Larger median kernels can remove dots, commas and thin strokes.
- Median 7×7 requires much more processing time.

4. Correlation, Convolution and Border Handling

Correlation applies the kernel without rotating it. Convolution rotates the kernel by 180° before applying it.

For the given 3×3 kernel:

Correlation result:

```
70   30  -10
190  190  70
270  230  110
```

Convolution result:

```
110  230  270
70   190  190
-10   30   70
```

These results match the supplied `cv2.filter2D()` comparison. OpenCV performs correlation by default. To perform convolution, the kernel must be flipped.

Border handling:

- Zero padding creates dark boundary artefacts.
- Reflect padding preserves the page boundary more naturally.

Therefore, `cv2.BORDER_REFLECT` is preferred.

5. Effect of Large Kernels and Missing Smoothing

Using a 7×7 Box Filter:

- Makes text strokes blurry.
- Weakens small characters and punctuation.
- Converts dust specks into grey patches.
- Produces a lower text Dice score of 0.51.

Skipping Smoothing Before Edge Detection:

- Grain and dust create unwanted edges.
- Canny detected 140,971 background edge pixels without smoothing.
- After 3×3 median filtering, this reduced to 1,470.

Thus, smoothing improves the reliability of edge detection and thresholding.

6. Classical Filter versus Pretrained Models

DnCNN is a deep-learning model that predicts and removes noise. Diffusion-based restoration models also attempt to restore noisy images.

Method	Advantages	Limitations
Median 3×3	Fast and preserves small text.	May not remove all fine grain.
DnCNN	Can remove noise while retaining details.	Requires a trained model and more processing time.
Diffusion Model	Can produce visually clean images.	May alter or generate incorrect letter strokes.

The pretrained models were not executed in the supplied experiment. Their performance therefore cannot be confirmed from measured results. For important document records, classical filtering is safer because it reduces the risk of changing original characters.

7. Final Decision

I select the Median Filter with a 3×3 kernel because:

- It achieved the highest PSNR among the tested filters (37.4 dB).
- It achieved an SSIM of 0.91.
- It preserved small text characters with a Dice score of 0.87.
- It required only about 2.8 ms per page.

Conclusion:

The 3×3 Median Filter provides the best balance between noise removal, text preservation and processing speed for scanned document images.

TASK 4 – Spatial Filtering and Image Sharpening

Compare Sharpening Filters

1. Introduction

Phone photographs may appear blurred, making letters difficult to read. Sharpening filters improve edge clarity by enhancing high-frequency details.

Three sharpening techniques are compared:

Technique	Basic Idea	OpenCV Functions
Laplacian Sharpening	Uses the second derivative to enhance edges.	cv2.Laplacian()
Unsharp Masking	Adds image details obtained by subtracting a blurred image.	cv2.GaussianBlur(), cv2.addWeighted()
Gradient Sharpening	Uses first-derivative Sobel gradients to strengthen edges.	cv2.Sobel()

2. Implementation

A. Laplacian Sharpening:

Formula: $g = f - c \times \text{Laplacian}(f)$

B. Unsharp Masking / High-Boost:

Formula: $g = f + k(f - \text{blurred image})$

Here, $k = 1$ represents unsharp masking and $k > 1$ represents high-boost filtering.

C. Gradient Sharpening:

Uses Sobel gradients in horizontal and vertical directions to enhance edges.

Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("blurred.jpg", 0)

lap = cv2.Laplacian(img, cv2.CV_32F)
A = np.clip(img.astype(float) - lap, 0, 255).astype(np.uint8)

blur = cv2.GaussianBlur(img, (0, 0), 2)
B = cv2.addWeighted(img, 2, blur, -1, 0)

x = cv2.Sobel(img, cv2.CV_32F, 1, 0)
y = cv2.Sobel(img, cv2.CV_32F, 0, 1)
mag = cv2.magnitude(x, y)
C = np.clip(img.astype(float) - 0.5*mag/4, 0, 255).astype(np.uint8)

images = [img, A, B, C]
titles = ["Original", "Laplacian", "Unsharp", "Gradient"]

for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Comparison of Sharpness and Noise

Four blurred pages were tested with blur $\sigma = 1.6$ and grain $\sigma = 3$. Sharpness was measured using variance of the Laplacian, while blank-region noise and OCR character error rate (CER) were also evaluated.

Filter	Sharpness	Noise σ	Halo	OCR CER	Time (ms)
Blurred input	183	3.1	-5.8	7.5%	0.0
Laplacian	18,979	28.3	1.6	135.7%	29.7
Unsharp (k=1)	731	6.0	-3.3	206.1%	8.9
High-boost (k=2.5)	2,241	10.4	0.5	188.2%	13.8
Gradient	197	3.2	-8.1	2.4%	68.4
Unsharp after median 3×3	51	2.4	-4.4	2.7%	14.0

Observations:

- Laplacian sharpening produces strong edges but greatly amplifies noise.
- Unsharp masking improves letter clarity but can create bright halos.
- High-boost filtering increases both sharpness and background noise.
- Gradient sharpening produces a low OCR error but provides limited sharpening.
- Smoothing before sharpening reduces noise and improves OCR results.

4. Stroke Profiles and Second-Derivative Analysis

A grayscale intensity profile across letter strokes helps identify changes in edge sharpness.

- First derivatives measure the rate of intensity change.
- Second derivatives measure changes in the rate of intensity change.
- For a spatial frequency ω, the first derivative scales with ω, while the second derivative scales with ω^2.
- Therefore, second derivatives emphasize fine details more strongly, but also amplify high-frequency noise.

The Laplacian produced a sharpness value of 18,979 compared with 197 for gradient sharpening. However, its blank-region noise increased from 3.1 to 28.3.

Conclusion: Sharpening should be performed after smoothing to prevent grain and dust from becoming stronger.

5. Effects of High-Boost and Unscaled Laplacian

High-boost with k = 5:

- Amplifies noise in an unsmoothed photograph.
- Background noise increased from $\sigma = 12$ to 45.
- Creates unwanted grain and rough-looking text edges.

Unscaled Laplacian:

- Contains both positive and negative values.
- Approximately 48.5% of the values were negative in the supplied experiment.
- Clipping negative values to zero removes important edge information.
- Scaling the output or adding the Laplacian back to the original image gives a more useful result.

6. Deep Learning Deblurring versus Unsharp Masking

Pretrained models such as NAFNet, Restormer and Real-ESRGAN can restore blurred images using learned features.

Method	Advantages	Limitations
Unsharp Masking	Simple, fast and easy to control.	Cannot fully restore severely blurred text.
Deep Deblurring	May improve heavily blurred text.	Requires a pretrained model and more processing time.
Super-Resolution	Can improve image resolution.	May generate or alter text details.

The pretrained models were not executed in the supplied experiment. Therefore, their OCR accuracy and processing time cannot be confirmed. Any restored document should be checked for altered characters before use.

7. Final Decision

I select Unsharp Masking ($k = 1$, $\sigma = 2$) after a 3×3 Median Filter.

Reasons:

- Reduces background noise to $\sigma = 2.4$.
- Achieves an OCR CER of approximately 2.7%.
- Improves text clarity while limiting unwanted noise.
- Provides controllable sharpening with a simple parameter.

The Laplacian amplified noise significantly, while gradient sharpening provided limited improvement in sharpness.

Conclusion:

The sequence of median smoothing followed by unsharp masking provides a suitable balance between noise reduction, text clarity and OCR performance.

TASK 5 – Image Enhancement in the Frequency Domain: Compare Low-Pass Filters

1. Introduction

Scanned pages may contain grain and horizontal scanner stripes. Frequency-domain filtering removes unwanted frequency components while preserving important image details.

The frequency-domain process is:

Padding → DFT → Frequency Shifting → Filtering → Inverse DFT → Cropping

Three low-pass filters are compared:

Filter	Basic Idea
Ideal Low-Pass Filter (ILPF)	Passes frequencies inside a fixed cut-off radius and removes others.
Butterworth Low-Pass Filter (BLPF)	Uses a smooth transition controlled by order n.
Gaussian Low-Pass Filter (GLPF)	Uses a Gaussian function for smooth filtering.

2. Implementation

The filter transfer functions are:

- ILPF: $H = 1$ when $D \leq D_0$, otherwise 0.
- BLPF: $H = 1 / [1 + (D/D_0)^{2n}]$.
- GLPF: $H = \exp(-D^2/2D_0^2)$.

Python Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("page.jpg", 0).astype(np.float32)

M, N = img.shape

p = cv2.copyMakeBorder(img, 0, M, 0, N, cv2.BORDER_REFLECT)

F = np.fft.fftshift(np.fft.fft2(p))


y, x = np.ogrid[:2*M, :2*N]

D = np.sqrt((y-M)**2 + (x-N)**2)

D0 = 160

filters = [

    (D <= D0),

    1 / (1 + (D/D0)**4),

    np.exp(-D**2 / (2*D0**2))

]


for H in filters:

    result = np.real(np.fft.ifft2(

        np.fft.ifftshift(F * H)))

    result = np.clip(result[:M, :N], 0, 255)

    plt.imshow(result, cmap="gray")

    plt.axis("off")

    plt.show()
```

3. Spectrum Analysis and Comparison

The scanner stripes repeat every 6 rows. Therefore, they appear as two bright frequency components at ±1/6 cycles per pixel in the centred spectrum.

The following results compare the filters at three cut-off frequencies.

Filter	D_0	Ringing (%)	OCR CER	PSNR (dB)
ILPF	80	8.9	81.4%	30.9
BLPF	80	3.5	76.0%	31.3
GLPF	80	0.0	72.9%	31.5
ILPF	160	10.7	68.9%	32.0
BLPF	160	4.4	13.3%	32.5
GLPF	160	1.2	1.9%	33.0
ILPF	320	0.0	4.2%	34.2
BLPF	320	1.8	1.4%	35.0
GLPF	320	0.9	1.7%	35.4

Values retained from the supplied experiment.

Observations:

- Ideal filtering produces ringing because of its abrupt cut-off.
- Butterworth reduces ringing using a smoother transition.
- Gaussian filtering produces almost no ringing.
- $D_0 = 80$ removes detail and causes poor OCR.
- $D_0 = 160$ provides a good balance between stripe removal and text preservation.

4. Frequency-Domain and Spatial-Domain Gaussian Filtering

Frequency-domain Gaussian filtering is equivalent to spatial Gaussian filtering when their parameters are matched.

For the supplied experiment:

- $D_0 = 160$
- Equivalent spatial $\sigma = 1.59$ pixels.
- Maximum output difference = 0.0003 grey levels.
- Mean difference = 0.000017.

The frequency-domain method was slower on a CPU for the tested page sizes.

Spatial Kernel	Spatial Time (ms)	FFT Time (ms)
5×5	21	7716
15×15	181	7716
31×31	186	7716
61×61	252	7716
121×121	489	7716

Frequency-domain filtering is more useful for specialized filters such as notch filtering.

5. Effects of Padding and Abrupt Cut-Off

Without Padding:

- The DFT treats the image as periodic.
- Image boundaries may wrap around.
- The top 15 rows became 26 grey levels darker in the supplied experiment.

Ideal Low-Pass Filter:

- Produces ringing near text edges.
- Creates unwanted ripples.
- Can reduce text quality and OCR accuracy.

Padding and smooth frequency transitions help reduce these problems.

6. Final Decision

I select the Gaussian Low-Pass Filter with $D_0 = 160$.

Reasons:

- Removes scanner stripes effectively.
- Produces only 1.2% ringing.
- Achieves an OCR CER of 1.9%.
- Preserves text better than the other filters at this cut-off.

A narrow notch filter can also be considered because it removes the specific scanner-stripe frequency while preserving other image details.

Conclusion

Gaussian low-pass filtering with $D_0 = 160$ provides a good balance between noise removal, text preservation and OCR accuracy.

TASK 6 – Image Enhancement in the Frequency Domain: Compare High-Pass Filters

1. Introduction

Phone photographs taken near windows may contain shadows that make one side of a page darker. Shadows are associated with low-frequency components, while letter edges contain high-frequency components.

High-pass filtering reduces low frequencies and preserves or enhances high frequencies.

Three approaches are compared:

Technique	Basic Idea
Ideal / Butterworth HPF	Removes low-frequency components.
High-Frequency Emphasis (HFE)	Preserves some low frequencies and boosts high frequencies.
Homomorphic Filtering	Separates illumination and reflectance to reduce shadows.

2. Implementation

The homomorphic method uses the illumination-reflectance model:

$f(x,y) = i(x,y) \times r(x,y)$

Taking logarithms converts multiplication into addition:

$\log(f) = \log(i) + \log(r)$

The low-frequency illumination is reduced, while high-frequency text details are strengthened.

Python Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("shadow.jpg", 0).astype(np.float32)

M, N = img.shape

F = np.fft.fftshift(np.fft.fft2(np.log1p(img)))


y, x = np.ogrid[:M, :N]

D = np.sqrt((y-M/2)**2 + (x-N/2)**2)

D0 = 20


H = 1.5 - (1.5-0.5)*np.exp(-D**2/D0**2)

g = np.expm1(np.real(np.fft.ifft2(

    np.fft.ifftshift(F*H))))


g = cv2.normalize(g, None, 0, 255,

                  cv2.NORM_MINMAX).astype(np.uint8)


plt.imshow(g, cmap="gray")

plt.axis("off")

plt.show()
```

3. Comparison of Contrast, Brightness and Noise

Three shadowed pages and two normally illuminated pages were used in the supplied experiment.

Technique	Shadow Contrast	Lit Contrast	Paper Unevenness	Mean Brightness	Noise
Input	17.5	50.5	44.4	171	27.1
Ideal HPF	33.4	52.9	2.6	187	37.5
Butterworth HPF	33.5	53.1	2.4	189	38.0
Gaussian HFE	42.0	79.8	22.0	85	53.8
Homomorphic	41.2	61.5	25.0	167	42.2
Homomorphic (swapped gains)	2.8	42.8	65.4	191	17.1

Values retained from the supplied experiment.

Observations:

- Ideal and Butterworth HPF reduce shadow unevenness but remove the DC component.
- High-frequency emphasis improves text contrast but increases noise.
- Homomorphic filtering improves shadow-side contrast while maintaining average brightness.
- Swapping the low- and high-frequency gains produces poor results.

4. Frequency-Domain and Spatial-Domain Laplacian

The frequency-domain Laplacian uses:

$$H(u,v) = -4\pi^2(u^2 + v^2)$$

The supplied experiment reported a correlation of 0.983 between frequency-domain and spatial-domain Laplacian outputs.

Both methods enhance edges but can also amplify noise.

5. Effects of Incorrect Filtering

Unmodified Ideal HPF:

- Removes the average brightness of the page.
- Loses paper tone and overall tonal information.
- Produces outlines instead of a normal document image.
- Can reduce OCR performance.

Interchanging γL and γH:

- Amplifies illumination variations.
- Suppresses important text details.
- Reduces shadow-side text contrast to 2.8.
- Increases paper unevenness to 65.4.

6. Homomorphic Filtering versus Deep Learning

Pretrained shadow-removal models such as BEDSR-Net can learn to remove shadows from document photographs.

Method	Advantages	Limitations
Homomorphic Filtering	Simple, controllable and does not require a trained model.	May not remove complex shadows completely.
Deep Learning Model	May handle complex shadows and uneven illumination.	Requires a pretrained model and more processing resources.

The pretrained model was not executed in the supplied experiment. Therefore, its OCR accuracy and processing time cannot be confirmed.

7. Final Decision

I select Homomorphic Filtering with $\gamma L = 0.5$, $\gamma H = 1.5$ and $D_0 = 20$.

Reasons:

- Improves shadow-side text contrast from 17.5 to 41.2.
- Reduces paper unevenness from 44.4 to 25.0.
- Preserves average brightness better than plain high-pass filtering.
- Provides a suitable balance between shadow correction and text preservation.

Conclusion

Homomorphic filtering is suitable for shadowed document photographs because it reduces illumination variations while preserving important text details.

TASK 7 – Image Segmentation

Compare Point, Line and Edge Detection

1. Introduction

Image segmentation identifies important structures by detecting sudden changes in pixel intensity.

In document images:

- Dust specks behave like isolated points.
- Ruled and table borders behave like lines.
- Page boundaries behave like edges.

Three detection approaches are compared:

Technique	Basic Idea	OpenCV Functions
Point and Line Detection	Uses Laplacian and directional masks.	cv2.filter2D()
Gradient Edge Detection	Uses Roberts, Prewitt and Sobel operators.	cv2.Sobel()
Advanced Edge Detection	Uses LoG zero-crossings and Canny.	cv2.Canny()

2. Implementation

Point detection identifies isolated dust specks using a Laplacian mask. Line detection uses directional masks. Edge detection identifies boundaries using gradient magnitude.

Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("page.jpg", 0)
smooth = cv2.medianBlur(img, 3)

# Point detection
point = cv2.Laplacian(smooth, cv2.CV_32F)
points = (point < -200).astype(np.uint8)

# Line detection
kernel = np.array([[-1,-1,-1],
                   [2,2,2],
                   [-1,-1,-1]], np.float32)
lines = cv2.filter2D(smooth, -1, kernel)

# Edge detection
edges = cv2.Canny(
    cv2.GaussianBlur(img, (5,5), 0), 60, 150)

images = [img, points, lines, edges]
titles = ["Original", "Points", "Lines", "Edges"]

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Precision and Recall Comparison

Precision and recall were calculated using hand-annotated ground-truth images with a 2-pixel matching tolerance.

Point Detection

Threshold T	Precision	Recall	F1
100	0.00	0.90	0.00
200	0.51	0.93	0.66
300	0.31	0.93	0.47
400	0.36	0.93	0.52
500	0.51	0.90	0.65
600	1.00	0.45	0.62
800	1.00	0.28	0.43

The best F1 score was 0.66 at T = 200.

Line Detection

Threshold T	Precision	Recall
20	0.96	1.00
40	0.98	0.72
60	1.00	0.17
80	0.00	0.00

A threshold of 20 detected all ruled lines with 96% precision.

Page-Border Detection

Detector	Precision	Recall
Roberts	0.84	1.00
Prewitt	0.85	1.00
Sobel	0.85	1.00
LoG	0.02	0.99
Canny	0.28	1.00

Sobel provided a good balance between precision and recall.

4. Threshold Selection and Observations

- Point detection: T = 200 gives the best F1 score.
- Line detection: T = 20 detects faint ruled lines effectively.
- Canny: thresholds 60 and 150 provide continuous page edges.

Tilted Pages:

Directional line masks may fail when pages are tilted because the lines no longer match the selected mask directions. Adding ±45° masks or deskewing the page improves detection.

5. Effects of Noise and Incorrect Detection

Edge Detection on Noisy Images:

- Noise creates unwanted edges.
- The supplied experiment detected 314,249 false edge pixels on blank paper.
- After median and Gaussian smoothing, this reduced to 60.

Treating Every Letter as a Point:

- Letters may be incorrectly identified as isolated points.
- The experiment reported 1,192 false point detections for only 7 real specks.
- This increases unnecessary manual checking.

6. Canny versus Learned Edge Detection

Holistically-Nested Edge Detection (HED) is a deep-learning-based method that predicts edge probabilities.

Method	Advantages	Limitations
Canny	Fast and suitable for edge detection.	May detect text strokes and background details.
HED	Can identify meaningful boundaries in cluttered images.	Requires a pretrained model and more processing time.

HED was not executed in the supplied experiment, so its expected performance cannot be confirmed through measured results.

7. Final Decision

I select different detectors for different structures:

- Dust specks: Laplacian point mask after 3×3 median filtering, T = 200.
- Ruled and table lines: Directional line masks with T = 20.
- Page borders: Sobel edge detection with a suitable threshold.

Canny is useful for general edge detection and can be combined with the Hough Transform for page-boundary detection.

Conclusion

Combining point, line and edge detection techniques provides more reliable document segmentation than using a single detector for every structure.

TASK 8 – Image Segmentation

Compare Thresholding Techniques

1. Introduction

Thresholding converts a grayscale document image into a binary image containing black text and white background. It is useful for document printing and OCR.

Three techniques are compared:

- Basic Global Thresholding: Calculates a threshold by repeatedly finding the means of two pixel groups.
- Otsu's Thresholding: Automatically selects the threshold that maximizes between-class variance.
- Adaptive Thresholding: Calculates a local threshold for each pixel using its neighbourhood.

2. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("page1.jpg", 0)

# A. Basic global threshold
T = img.mean()
while True:
    a, b = img[img <= T].mean(), img[img > T].mean()
    new_T = (a + b) / 2
    if abs(new_T - T) < 0.5:
        break
    T = new_T
A = cv2.threshold(img, T, 255, cv2.THRESH_BINARY_INV)[1]
```

```
# B. Otsu thresholding
_, B = cv2.threshold(img, 0, 255,
                     cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# C. Adaptive Gaussian thresholding
C = cv2.adaptiveThreshold(cv2.medianBlur(img, 3), 255,
                          cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                          cv2.THRESH_BINARY_INV, 51, 12)

for i, (x, name) in enumerate(zip(
        [img, A, B, C],
        ["Original", "Global", "Otsu", "Adaptive"])):
    plt.subplot(2, 2, i + 1)
    plt.imshow(x, cmap="gray")
    plt.title(name)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Performance Comparison

The following results are supplied in the experiment.

Technique	Mean Dice	Worst Page	Shadowed Text Kept	Stain Area Black	Time/Page
Basic Global	0.38	0.01	99.5%	54.0%	305 ms
Otsu	0.23	0.01	99.5%	53.8%	12 ms
Otsu + Gaussian 5×5	0.37	0.00	100%	53.6%	14 ms
Adaptive Gaussian	0.84	0.73	58.3%	12.8%	104 ms
Adaptive Mean	0.83	0.73	68.3%	21.2%	24 ms

Observation:

Adaptive Gaussian provides the highest mean Dice score and lowest stain-related black regions. Adaptive Mean is faster and preserves more shadowed text, but produces more stain artefacts.

4. Illumination–Reflectance Model

The image formation model is:

[f(x,y)=i(x,y)\times r(x,y)]

Where:

- (f(x,y)) = observed image.
- (i(x,y)) = illumination.
- (r(x,y)) = reflectance of the document.

Under uneven lighting, the shadow reduces illumination. For example, shadowed paper may have an intensity of approximately 83, while lit text may have an intensity of 140.

Therefore, a single global threshold cannot correctly separate text and paper throughout the image. Adaptive thresholding handles this problem by calculating thresholds from local neighbourhoods.

Effect of Gaussian Smoothing on Otsu

Gaussian smoothing reduces small noise before threshold selection.

Method	Mean Dice
Otsu	0.23
Otsu after Gaussian 5×5	0.37

The mean Dice increased from 0.23 to 0.37. Therefore, Gaussian smoothing improves Otsu's performance, but it cannot completely correct uneven illumination.

5. Parameter Sensitivity and Failure Cases

Adaptive Gaussian thresholding was tested using different block sizes and constant values.

Block Size	C = 5	C = 12	C = 20
5	0.68	0.23	0.01
11	0.56	0.71	0.32
25	0.51	0.82	0.61
51	0.52	0.84	0.69
101	0.54	0.83	0.73

A block size of 5 pixels is smaller than the character height of approximately 25 pixels. It can cause characters to become hollow, broken or fragmented.

Failure cases:

- Otsu thresholding may classify shadowed paper as black foreground.
- A very small adaptive block size may break character strokes and create false foreground regions.
- A suitable block size must be larger than the character dimensions.

6. Classical Method vs Deep Learning

A pretrained DIBCO-trained U-Net can learn to separate text from stains and background noise.

However, the deep-learning model was not executed in this experiment. Its performance is therefore not measured or directly compared.

Adaptive Gaussian thresholding achieved a supplied mean Dice score of 0.84 and processed a page

TASK 9 – Region-Based Segmentation, Edge Linking and Boundary Detection

1. Introduction

Region-based segmentation identifies complete regions such as stains and ink blots. Boundary detection identifies the four edges of a document so that the page can be cropped correctly.

Three techniques are compared:

- A. Region Growing: Starts from a seed pixel and adds similar neighbouring pixels.
- B. Region Splitting and Merging: Divides the image into smaller regions and combines similar regions.
- C. Edge Linking: Connects edge pixels, closes gaps and detects boundaries using contours and the Hough transform.

2. Python Implementation

```
import cv2
import numpy as np

img = cv2.imread("page1.jpg", 0)
f = cv2.medianBlur(cv2.medianBlur(img, 7), 7)

# A. Region growing
mask = np.zeros((img.shape[0]+2, img.shape[1]+2), np.uint8)
cv2.floodFill(f.copy(), mask, (100, 100), 0, 10, 10,
        8 | cv2.FLOODFILL_FIXED_RANGE |
        cv2.FLOODFILL_MASK_ONLY | (255 << 8))
A = mask[1:-1, 1:-1]

# B. Region splitting and merging (simple region mask)
B = cv2.threshold(f, int(np.median(f))-14, 255,
            cv2.THRESH_BINARY_INV)[1]
```

```
# C. Edge linking and contours
edges = cv2.Canny(f, 10, 30)
edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE,
                         np.ones((9, 9), np.uint8))
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
C = np.zeros_like(img)
cv2.drawContours(C, contours, -1, 255, -1)

for name, result in zip(["Region Growing", "Splitting", "Edge Linking"],
                        [A, B, C]):
    cv2.imshow(name, result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

3. Performance Comparison

The following results are supplied for 10 stained pages and 10 phone photographs.

Approach	Stain IoU	Median Corner Error	Worst Corner Error	Stain Time
A. Region Growing	0.68	2.1 px	2.5 px	256 ms
B. Split and Merge	0.42	6.8 px	96.3 px	173 ms
C. Edge Linking + Hough	0.89	6.3 px	601.1 px	141 ms

Observations:

- Region growing detects stains but may miss their darker outer boundaries.
- Split and merge struggles when text inside a stain makes the region non-uniform.
- Edge linking gives the highest stain IoU of 0.89.

- Region growing gives the most accurate page corners, with a median error of 2.1 pixels.
- Hough detection may fail when a stain edge is mistaken for a page boundary.

4. Integrated Smart-Scanner Pipeline

The selected processing stages from Tasks 1–8 are combined with Task 9 segmentation.

Pipeline flow:

Original Document Image

↓

Grayscale Conversion / CLAHE

↓

Median Filtering

↓

Optional Frequency Filtering for Stripes

↓

Page Boundary Detection

↓

Perspective Correction

↓

Adaptive Gaussian Thresholding

↓

Stain Detection using Edge Linking

↓

Stain Annotation and Final Page Cropping

↓

Processed Document Output

Integrated Pipeline Code

```
import cv2
import time

def scan_pipeline(gray):
    start = time.perf_counter()

    g = cv2.createCLAHE(clipLimit=4, tileGridSize=(16,16)).apply(gray)
    g = cv2.medianBlur(g, 3)

    edges = cv2.Canny(g, 30, 90)
    lines = cv2.HoughLines(edges, 1, 3.14/180, 200)

    binary = cv2.adaptiveThreshold(
        g, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, 51, 12)

    stain_edges = cv2.Canny(cv2.medianBlur(g, 7), 10, 30)
    stain_edges = cv2.morphologyEx(
        stain_edges, cv2.MORPH_CLOSE, np.ones((9,9), np.uint8))

    total = time.perf_counter() - start
    return binary, stain_edges, lines, total
```

Processing Time

The supplied experiment reports a total processing time of 1,456 ms (1.46 seconds) for a 3348 × 4385 image.

The slowest stage was stain processing, which took 1,197 ms in the timed implementation.

Therefore, the measured pipeline satisfies the required 3-second limit.

Important:

The timed implementation used Hough detection for page boundaries and split-and-merge for stains. The final recommended pipeline replaces these with region growing for page detection and edge linking for stains. The estimated time is expected to remain below 1.46 seconds, but the modified pipeline must be timed again to confirm this.

5. Design Decision Table

Decision	Evidence-Based Justification	Consequence if Not Selected
Median 3×3 filtering	Reduces false edges from 140,971 to 1,470	Dust may be detected as text
CLAHE (4.0, 16×16)	Improves illumination evenness	Shadowed text remains unclear
Gaussian LPF or notch filter	Reduces scanner stripes	OCR errors may increase
Homomorphic filtering for heavy shadows	Improves shadow contrast	Global thresholding may fail
Adaptive Gaussian threshold (51, 12)	Dice 0.84 in Task 8	Shadowed paper may become black
Region growing for page detection	Median corner error 2.1 px	Incorrect page cropping
Edge linking for stains	Stain IoU 0.89	Stains may be missed

6. Predictions from Tasks 1–8

Supported predictions:

- Median filtering performs better than linear filters for removing dust.
- Adaptive thresholding performs better than global thresholding under shadows.
- Gaussian low-pass filtering reduces ringing effects.

Contradicted predictions:

- Sharpening does not always improve OCR. On noisy images, it can increase errors unless median filtering is applied first.
- Frequency-domain filtering was expected to be faster, but the full-page NumPy FFT took 7.7 seconds, which was slower than spatial filtering.

The large image size and FFT processing overhead caused this difference.

7. Failure Cases and Corrective Measures

A. Seed Placed on a Text Character

If the seed is placed on a letter instead of a stain, region growing follows the character stroke rather than the stain.

The supplied experiment reported a stain IoU of only 0.0001.

Correction: Select seeds from large, low-texture dark regions. Avoid thin character strokes.

B. Interrupted Page Boundary

A deep shadow can interrupt the page boundary and cause Hough detection to identify incorrect corners.

The supplied corner error was 868 pixels. Applying CLAHE before Canny reduced it to 240 pixels.

Correction:

- Apply CLAHE before edge detection.
- Lower the Hough voting threshold when a side is missing.
- Check whether opposite page sides are approximately parallel.
- Use region growing as a fallback

.

8. Region Growing vs Watershed vs SAM 2

Method	Stain IoU	Main Observation
Region Growing	0.68	Requires seed selection
Watershed	0.49	Lower stain detection accuracy
Edge Linking	0.89	Best automatic stain detection
SAM 2	Not measured	Requires prompts and model execution

SAM 2 is a promptable segmentation model that can identify regions using user-provided points. However, it was not executed in this experiment, so its accuracy and processing time are not measured.

It may be useful for difficult stains and punch holes, while edge linking is more suitable for automatic batch processing.

9. Final Decision

I select the following approaches:

- Stain Detection: Edge Linking and Contours (IoU 0.89).
- Page Corner Detection: Region Growing (median error 2.1 px), with Hough detection as a cross-check.
- Final Thresholding: Adaptive Gaussian Thresholding (block size 51, C = 12).
- Integrated Pipeline: The measured version processed a full page in 1.46 seconds, within the 3-second requirement.

Therefore, combining region-based segmentation, edge linking and adaptive thresholding provides an effective smart-scanner solution.

Note: Numerical results are reproduced from the supplied experiment and have not been independently verified by executing the code. The final replacement pipeline requires fresh timing, and SAM 2 was not executed.

TASK 10 – Re-apply and Adapt the Smart Scanner Pipeline

1. Introduction

The original smart scanner was designed for flatbed document scans. The new requirement involves 12-megapixel colour JPEG photographs (4000 × 3000 pixels) captured using mobile phones under classroom tube lights.

The new challenges are:

- Uneven illumination and shadows.
- Tilted pages and perspective distortion.
- Blue handwritten notes that must be preserved.
- JPEG compression patterns (8 × 8 blocks).
- Processing time reduced from 3 seconds to 1 second per page.

Three strategies are evaluated:

- A. Retain: Apply the original Task 9 pipeline without modification.
- B. Modify: Resize images, select a suitable colour channel and adjust processing parameters.
- C. Redesign: Apply homomorphic filtering, JPEG-block smoothing, Hough detection and perspective correction.

2. Implementation of Three Strategies

A test set of 15 phone-style document images was prepared with shadows, tilt, blue annotations and JPEG compression.

A. Retain Strategy

The original grayscale pipeline is applied at full resolution.

```
import cv2

def strategy_A(img):
```

```
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(4.0, (16,16))
    g = cv2.medianBlur(clahe.apply(gray), 3)
    b = cv2.adaptiveThreshold(g, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 51, 12)
    return b
```

B. Modify Strategy

The image is resized to 1500 pixels in width. The red channel is selected because blue pen marks appear darker in this channel. CLAHE and adaptive threshold parameters are adjusted.

```
def strategy_B(img):
    s = 1500 / img.shape[1]
    small = cv2.resize(img, None, fx=s, fy=s,
                       interpolation=cv2.INTER_AREA)
    red = small[:, :, 2]
    g = cv2.createCLAHE(3.0, (8,8)).apply(red)
    g = cv2.medianBlur(g, 3)
    b = cv2.adaptiveThreshold(g, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 25, 10)
    return b
```

C. Redesign Strategy

Homomorphic filtering is applied to reduce uneven illumination. Light Gaussian smoothing reduces JPEG compression artefacts before thresholding.

```
def strategy_C(img):
    small = cv2.resize(img, (1500, 2000))
    red = cv2.GaussianBlur(small[:,:,2], (3,3), 0)
    h = homomorphic(red, 0.4, 1.4, d0=15)
    b = cv2.adaptiveThreshold(h, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 25, 10)
    return b
```

3. Original Pipeline Assumptions and Performance Audit

Table 21: Changed Assumptions

Original Assumption	New Requirement	Consequence
2480 × 3508 grayscale scan	4000 × 3000 colour JPEG	Higher processing cost
Grayscale is sufficient	Blue notes must be preserved	Some annotations become difficult to distinguish
Flat page	Angled phone photograph	Perspective correction is required
Scanner noise and stripes	JPEG 8 × 8 blocks and vignetting	Noise characteristics change
3-second processing	1-second processing	Full-resolution processing may be too slow

Stages That Remain Effective

- Median filtering reduces small noise.
- CLAHE improves local contrast.
- Adaptive thresholding handles uneven illumination.
- Hough detection helps identify page boundaries.

Stages Requiring Modification

- Resize images before processing.
- Use the red channel to preserve blue handwritten notes in the binary result.
- Remove unnecessary scanner-stripe filtering.
- Re-tune thresholding and CLAHE parameters.
- Apply perspective correction to the original colour image.

4. Comparison of Strategies A, B and C

The following results are supplied from the 15-image experiment.

Measured Performance

Strategy	Text Dice	Blue Notes Kept	Median Corner Error	Time/Page	Under 1 Second?
A. Retain	0.62	100.0%	479 px	327 ms	Yes
B. Modify	0.79	99.7%	24 px	119 ms	Yes
C. Redesign	0.76	99.3%	24 px	2134 ms	No

Observations

Strategy A:

- Processing time is 327 ms.
- Text Dice is only 0.62.
- Corner error is 479 pixels.
- Original parameters are unsuitable for the new image dimensions.

Strategy B:

- Achieves the highest Text Dice of 0.79.
- Preserves 99.7% of blue notes.
- Reduces corner error to 24 pixels.
- Processes a page in 119 ms.

Strategy C:

- Achieves a Text Dice of 0.76.
- Preserves 99.3% of blue notes.
- Requires 2134 ms per page.
- Exceeds the 1-second processing limit.

Therefore, the ranking is:

B (Modify) > C (Redesign) > A (Retain).

5. Revised Smart-Scanner Pipeline

The selected pipeline is:

Original Colour JPEG

↓

Resize to 1500 Pixels Width

↓

Red Channel Extraction

↓

CLAHE (3.0, 8 × 8)

↓

Median Filtering (3 × 3)

↓

Hough Page Boundary Detection

↓

Perspective Correction of Original Colour Image

↓

Adaptive Gaussian Thresholding (25, 10)

↓

Preserve Blue Annotations and Generate Binary Copy

↓

Final Document Output

The original colour image is retained for archival purposes, while the binary image is used for document processing.

Performance

The supplied experiment reports:

- Average processing time: 119 ms per page.
- Text Dice: 0.79.
- Blue-note preservation: 99.7%.
- Median corner error: 24 pixels.

The processing time is below the required 1000 ms limit.

Important: The supplied Strategy B timing must be measured again after adding perspective correction and full-resolution colour-image warping. The reported 119 ms should not automatically be treated as the timing of that complete revised pipeline.

6. Limitation and Failure Condition

The selected Strategy B may not work well under a strong, sharp-edged shadow crossing the text.

CLAHE with 8 × 8 tiles may not completely correct the shadow. Adaptive thresholding may also classify the shadow boundary as a false foreground region.

Corrective measure:

- Detect strong illumination changes.
- Apply homomorphic filtering to a reduced-resolution image only when necessary.
- Recheck the thresholded result before final output.

However, this additional processing must be timed to ensure that the 1-second requirement is maintained.

7. Document-AI Models: LayoutLMv3 and Donut

Document-AI models can assist with understanding document content and structure.

Component	Possible Model Role	Should Conventional Processing Remain?
Image enhancement	Improve document readability	Yes, for predictable image quality
Page detection	Locate document boundaries	Yes, for accurate cropping
Perspective correction	Correct tilted pages	Yes, for geometric accuracy
OCR	Extract printed and handwritten text	Can be assisted by AI-based OCR
Layout interpretation	Identify headings, tables and question numbers	LayoutLMv3 or Donut can assist
Structured information extraction	Convert document content into structured data	Donut can assist

LayoutLMv3: Combines text, image and layout information to understand document structure.

Donut: A document-understanding model that can generate structured information directly from document images without a separate OCR stage.

These models can assist with text recognition and layout interpretation. However, they do not automatically replace accurate page detection, geometric correction and colour preservation.

8. Final Decision – RETAIN / MODIFY / REDESIGN

I select **MODIFY (Strategy B)**.

Decision	Evidence	Reason
RETAIN – A	Dice 0.62, corner error 479 px	Fast but inaccurate
REDESIGN – C	Dice 0.76, time 2134 ms	Exceeds the 1-second limit
MODIFY – B	Dice 0.79, 99.7% blue-note preservation, 24 px corner error, 119 ms	Best measured accuracy and runtime

The modified pipeline provides the best balance between accuracy, annotation preservation and processing speed.

Therefore, resizing the image, selecting the red channel and adjusting CLAHE and adaptive threshold parameters provides a more suitable solution for mobile-phone document photographs.

Note:

The numerical results are reproduced from the supplied experiment and have not been independently verified. The 15-image dataset was generated using a simulation procedure rather than captured from 15 physical phones. The results therefore demonstrate performance on simulated phone photographs, not a verified real-world phone dataset. Full perspective-corrected pipeline timing and Document-AI model performance were not independently measured.